

Summary

Understanding how accurately people know what they know is among the most fundamental questions in cognitive neuroscience. `metasignal` is a Python package that makes measuring this capacity — metacognition — accessible to any researcher with a Python environment. It implements all twenty measures evaluated in the benchmark by @rahnev2025, spanning first-order perceptual sensitivity (d'), response bias (criterion c), and seventeen second-order metacognitive measures including meta- d' , M-ratio, meta-uncertainty, meta-noise, Type 2 AUC, gamma, phi, and delta confidence. The full suite is computable from a single function call, with no proprietary software required.

Statement of Need

Metacognition shapes learning, clinical outcomes, and adaptive decision-making across virtually every domain of cognition [flemingdolan2012]. A 26-researcher consensus initiative identified developing falsifiable computational models of visual metacognition as a primary long-term goal [rahnev2022] — an agenda that depends critically on reliable, standardised implementations of the measures used to evaluate those models. Yet the measurement toolkit has remained fragmented and inaccessible. The gold-standard measure, meta- d' [maniscalco2012], requires solving a constrained maximum-likelihood problem that has historically been available only as a MATLAB script [maniscalco2014], placing it out of reach for the growing majority of researchers working in Python. The broader landscape of metacognitive measures lacked any systematic comparison until @rahnev2025 — and even then, no open Python implementation of the full benchmark followed.

`metasignal` closes that gap with a single, maintained package that:

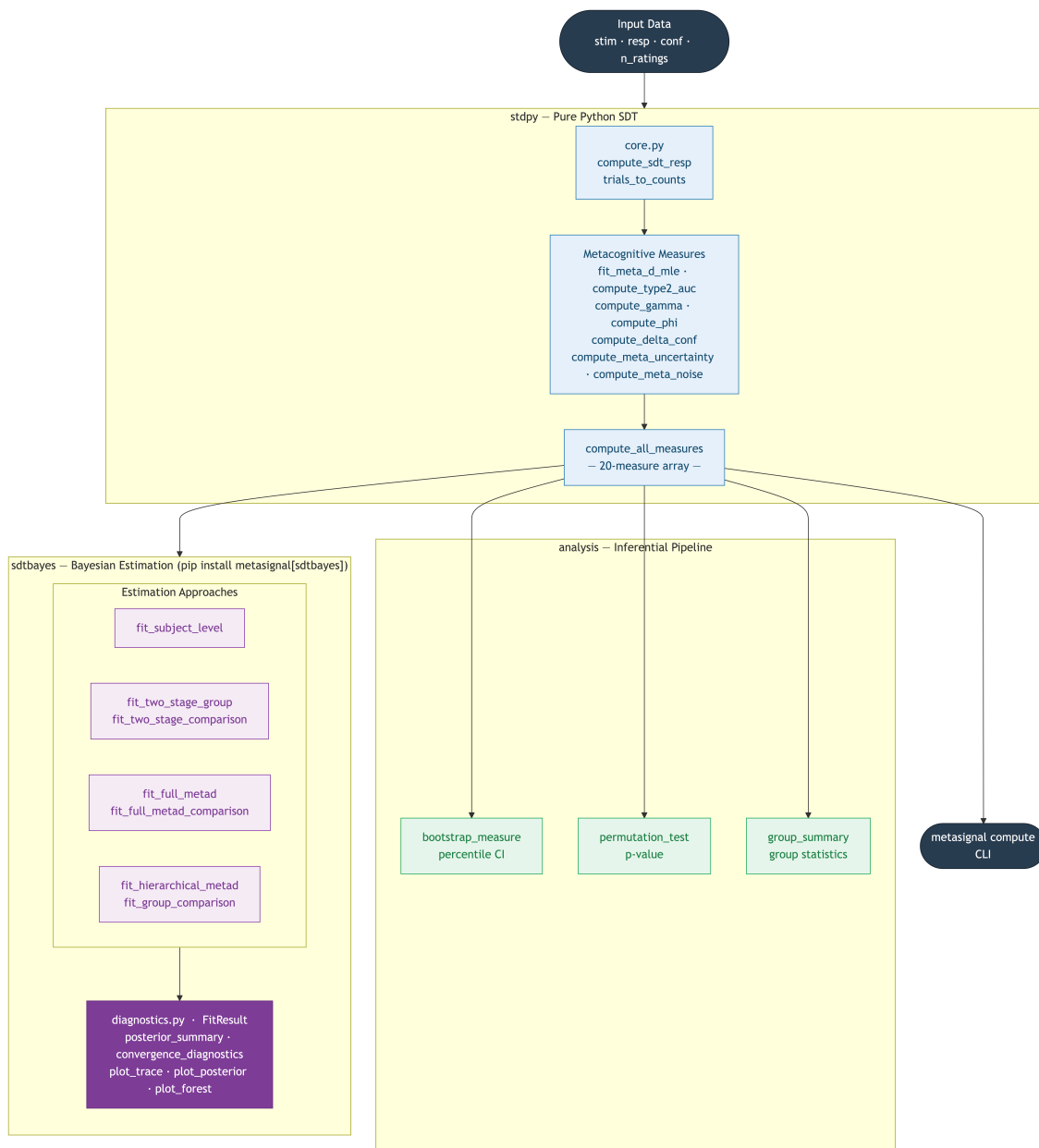
1. **Covers the full benchmark** — all twenty measures from @rahnev2025 (seventeen metacognitive measures plus d' , criterion c , and mean confidence as Type 1 reference values) are available through a single `compute_all_measures` call.
2. **Requires no proprietary software** — the `stdpy` submodule is a pure NumPy/SciPy implementation that runs in any Python environment, including cloud notebooks and automated pipelines.
3. **Lowens the barrier to entry** — a command-line interface lets researchers compute all twenty measures from raw trial data in a single shell command, without writing any Python code.

State of the Field

The canonical meta- d' implementation by @maniscalco2014 has been the field's workhorse since 2012 but is MATLAB-only and covers a single measure. `metadpy` [legrand2021] and `hmeta-d` [fleming2017] brought hierarchical Bayesian meta- d' to Python and R respectively, addressing estimation reliability but still covering only a fraction of the measures benchmarked by @rahnev2025. No existing package provides the meta-uncertainty and meta-noise estimators, the full suite of ratio and difference normalisation variants, a CLI-based workflow, or a unified interface that combines the full frequentist benchmark with hierarchical Bayesian estimation. The result is that researchers wishing to compare measures — as recommended by @rahnev2025 — have had to stitch together code from multiple repositories in multiple languages.

`metasignal` consolidates this fragmented landscape into a single, documented, tested Python package. Its optional `sdtbayes` subpackage further adds full hierarchical Bayesian inference — including a Stan port of the HMeta- d model of @fleming2017 — making it the only Python package that addresses both the breadth of the @rahnev2025 benchmark and the estimation reliability concerns raised by @fleming2017.

Software Design



`metasignal` accepts NumPy arrays of stimulus labels, responses, and confidence ratings as its universal input and exposes all measures through the `stdpy` submodule, implemented in NumPy [harris2020] and SciPy [virtanen2020]. Core estimation routines include:

- `compute_sdt_resp` — d' and criterion c from trial-level stimulus and response vectors.
- `trials_to_counts` — conversion of trial-level data into Type 2 rating-scale count matrices.
- `fit_meta_d_mle` — maximum-likelihood estimation of meta- d' and M-ratio via bounded nonlinear optimisation.
- `compute_type2_auc`, `compute_gamma`, `compute_phi`, `compute_delta_conf` — nonparametric Type 2 statistics.

- `compute_meta_uncertainty` — model-based meta-uncertainty estimation.
- `compute_meta_noise` — meta-noise estimation via lookup-table interpolation.
- `compute_all_measures` — computes all twenty measures in a single call, returning a fixed-length NumPy array whose index-to-measure mapping is documented in the API reference.

Beyond point estimation, the `metasignal.analysis` sub-package provides a complete inferential pipeline for group-level research. `bootstrap_measure` estimates percentile-bootstrap confidence intervals for any element of the twenty-measure array (default 2000 resamples, reproducible via an optional `numpy.random.Generator`), enabling uncertainty quantification without parametric assumptions. `permutation_test` implements a two-sided permutation test for between-condition differences (default 5000 shuffles), which is the recommended alternative to parametric t-tests given that the sampling distributions of measures such as M-ratio and meta-d' are non-Gaussian at typical sample sizes [Fleming2017; Rahnev2025]. `group_summary` aggregates results across participants, returning per-participant matrices, group means, medians, standard errors, and per-measure valid counts that handle NaN values arising from degenerate response patterns.

For researchers requiring full Bayesian inference, the optional `metasignal.sdtbayes` subpackage (installed via `pip install metasignal[sdtbayes]`) provides three hierarchical estimation approaches backed by brms [Burkner2017] and ArviZ [Kumar2019]:

- `fit_subject_level` — subject-level Bayesian SDT, yielding full posteriors over d' and criterion c for individual participants.
- `fit_two_stage_group` — a two-stage approach that first computes per-participant MLE M-ratios (Stage 1) then fits a hierarchical Bayesian model over log M-ratio across participants (Stage 2), providing a group-level posterior mean M-ratio with uncertainty.
- `fit_full_metad` — a full hierarchical HMeta-d model that ports the JAGS implementation of Fleming2017 to Stan, jointly estimating group-level and per-subject meta-d', M-ratio, d', and criterion from raw count matrices in a single pass.

Group comparison variants (`fit_two_stage_comparison`, `fit_full_metad_comparison`) extend each approach to two-group designs. All fits return ArviZ `InferenceData` objects, and the `diagnostics` module provides `posterior_summary`, `convergence_diagnostics`, `plot_trace`, `plot_posterior`, and `plot_forest` for routine MCMC quality checks. Together, `sdtbayes` makes `metasignal` the only Python package that covers both the full frequentist benchmark of Rahnev2025 and the hierarchical Bayesian estimation approach of Fleming2017 within a single, consistent interface.

The `metasignal compute` CLI prints all twenty measures as a named table from comma-separated trial data, enabling shell-level integration with experiment-control software and batch pipelines. Six tutorial Jupyter notebooks accompany the package, covering basic SDT computation, the full measure suite, statistical inference, difficulty-dependence testing, metacognitive bias, and split-half reliability — making the package accessible to researchers regardless of programming background.

AI Usage Disclosure

Claude (Anthropic) was used to assist in writing portions of this paper and in generating initial drafts of source code and documentation. All content was reviewed, verified, and edited by the author. The software logic, numerical methods, and validation against reference outputs are the intellectual contribution of the author.

References